PRACTICAL 3

Aim- 1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

1. **Kill processes by name**

```
vidhi@VidhiKore MSYS ~
$ cmd.exe /c "tasklist | findstr notepad"
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vidhi\Downloads\home\vidhi>
```

2. **Kill a process based on the process name**

```
vidhi@VidhiKore MINGW64 ~
$ cmd.exe /c "taskkill /F /IM chrome.exe"
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vidhi>tasklist | grep chrome.exe
chrome.exe                  13456 Console                    8   1,86,492 K
chrome.exe                  10784 Console                    8       3,920 K
chrome.exe                  16692 Console                    8   1,74,740 K
chrome.exe                   4612 Console                    8      41,444 K
chrome.exe                  13432 Console                    8      11,216 K
chrome.exe                  16076 Console                    8      38,328 K
chrome.exe                  17648 Console                    8   2,57,340 K
chrome.exe                  15540 Console                    8      48,256 K
chrome.exe                  18652 Console                    8      38,660 K
chrome.exe                  14848 Console                    8      76,840 K
chrome.exe                  14976 Console                    8      88,756 K
chrome.exe                  14412 Console                    8   2,88,324 K
chrome.exe                   4916 Console                    8      31,224 K

C:\Users\vidhi>
```

2. **Write a program for process creation using C**

- **Orphan Process OUTPUT:**

```
  GNU nano 8.7                                    orphan.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {

    pid_t pid = fork();

    if (pid > 0) {
        // Parent process
        printf("Parent process exiting...\n");
        printf("Parent PID: %d\n", getpid());
    }
    else if (pid == 0) {
        // Child process
        sleep(5);
        printf("Child process running...\n");
        printf("Child PID: %d\n", getpid());
        printf("New Parent PID (init/systemd): %d\n", getppid());
    }
    else {
        printf("Fork failed\n");
    }

    return 0;
}
```

```
vidhi@VidhiKore MSYS /c/Users/vidhi/Desktop
$ ./orphan
Parent process exiting...
Parent PID: 1213

vidhi@VidhiKore MSYS /c/Users/vidhi/Desktop
$ Child process running...
Child PID: 1214
New Parent PID: 1
```

- **Zombie Process**

```
  GNU nano 8.7                                    zombie.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid = fork();

    if (pid > 0)   // Parent
    {
        printf("Parent Process ID: %d\n", getpid());
        printf("Child PID (Zombie): %d\n", pid);
        sleep(15);   // parent does NOT wait
    }
    else if (pid == 0)  // Child
    {
        printf("Child Process ID: %d\n", getpid());
        printf("Child exiting...\n");
    }
    else
    {
        printf("Fork failed\n");
    }

    return 0;
}
```

OUTPUT:

```
vidhi@VidhiKore MSYS /c/Users/vidhi/Desktop
$ gcc zombie.c -o zombie

vidhi@VidhiKore MSYS /c/Users/vidhi/Desktop
$ ./zombie

PARENT RUNNING (PID: 1252)
Child PID: 1253

Now check process list using: ps
CHILD PROCESS (PID: 1253)
Child exiting immediately...

vidhi@VidhiKore MSYS /c/Users/vidhi/Desktop
$
```

```
~

vidhi@VidhiKore MSYS ~
$ ps | grep zombie
     1252     1089     1252      21580  pty2      197609 19:13:56 /c/Users/vidhi/Desktop/zombie

vidhi@VidhiKore MSYS ~
$
```

3. **Create the process using fork () system call.**

- **Child Process creation**
- **Parent process creation**
- **PPID and PID**

```
~
  GNU nano 8.7
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t pid = fork();   // ← yahan '=' missing tha

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());
    }
    else {
        // Parent process
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Child PID: %d\n", pid);
        wait(NULL);
    }

    return 0;
}
```

OUTPUT:

```
vidhi@VidhiKore MSYS /c/Users/vidhi/Desktop
$ gcc fork.c -o fork

vidhi@VidhiKore MSYS /c/Users/vidhi/Desktop
$ ./fork
Parent Process
Child Process
PID : 1222
Child PID: 1223
PID : 1223
PPID: 1222
```